Assignment: Adapter Design Pattern

Online Store Payment System

Scenario

An online store has a checkout module. The checkout code works with a single interface called `PaymentProcessor`:

- `String getName()` – name of the payment method.
- `boolean pay(String customer, double amountInDollars)` – returns `true` if the payment succeeded.
- `boolean refund(String customer, double amountInDollars)` – returns `true` if the refund succeeded.

The store now wants to accept payments through three existing third-party services. Their code **cannot be changed**, and their interfaces do not match `PaymentProcessor`:

Service	Pay method	Refund method	What is different
`LegacyBankGateway`	`int makeTransfer(String accountHolder, long amountInCents)`	`int reverseTransfer(String accountHolder, long amountInCents)`	Amounts are in **cents**. Returns a status code: `0` = success, anything else = failure. It declines transfers above 100000 cents.
`QuickPaySDK`	`String sendMoney(String currency, double amount, String payerName)`	`boolean returnMoney(String transactionId)`	Different parameter order and a **currency** code is needed (use `"USD"`). `sendMoney` returns a **transaction id** (for example `QP-1001`) or `null` on failure. A refund needs that transaction id, not the customer name.
`EuroPayService`	`boolean chargeCard(String customerEmail, double amountInEuros)`	`boolean refundCard(String customerEmail, double amountInEuros)`	Works in **euros** and identifies customers by **email**.

The store uses a fixed conversion rate of **1 USD = 0.90 EUR**. For EuroPay, build the email as the lower-case customer name followed by `@store.com` (for example `carol@store.com`).

The checkout code must keep calling `pay(...)` and `refund(...)` and must not know which service is used behind the interface.

Requirements

- Define the `PaymentProcessor` interface (the interface the client expects).
- Keep the three service classes exactly as given. Do not modify them.
- Make all three services usable wherever a `PaymentProcessor` is expected.
- Each adapter must convert the data (units, parameter order, identifiers) and the results (status codes, ids) into what the client expects.
- The `Checkout` (client) class must not contain any `if/else` or type checks to decide which service to call.
- `Checkout` must work with a **list** of different payment processors.

Tasks

Task 1 – Identify the pattern elements (20 marks)

Name the design pattern that fits this scenario and say why. Then fill in this table for your design:

Pattern role	Your class/interface	Responsibility
Target	?	?
Adaptee	?	?
Adapter	?	?
Client	?	?

Then answer briefly:

1. For each adapter, list exactly what it has to convert when the client calls `pay("Alice", 25.50)`.
2. Why does the QuickPay adapter need to store some data between calls?
3. There are two forms of the pattern: **object adapter** (composition) and **class adapter** (inheritance). Which one do you use, and why?

Task 2 – Design the UML diagram (30 marks)

Draw a **UML class diagram** (by hand or with a tool such as draw.io or PlantUML) showing all classes and interfaces, their attributes and operations, and the relationships between them. Show clearly that each adapter implements the Target and holds a reference to its Adaptee. The client should depend only on the Target.

Also draw a **UML sequence diagram** for `Checkout` calling `refund("Bob", 100.00)` through the QuickPay adapter.

Task 3 – Implement the design (50 marks)

Implement your design in any object-oriented language approved by your instructor. Your program must:

1. Contain the Target interface, the three Adaptees, and one Adapter for each Adaptee.
2. Have a `Checkout` (client) class that works with any `PaymentProcessor` and prints the result of each payment or refund.
3. Run the sample transactions below using the same client code for every service.
4. Handle failures correctly: a declined payment must return `false` and be reported by the client.
5. Avoid changing the Adaptee classes.

Sample Data

The three Adaptee classes print a short message when called. Process these transactions through `Checkout`:

#	Processor	Action	Customer	Amount (USD)	Expected result
1	Legacy bank	pay	Alice	25.50	Success (2550 cents)
2	QuickPay	pay	Bob	100.00	Success (transaction id QP-1001)
3	EuroPay	pay	Carol	50.00	Success (45.00 EUR)
4	Legacy bank	pay	Dave	1500.00	Failure (150000 cents is over the limit)
5	QuickPay	refund	Bob	100.00	Success (uses QP-1001)
6	EuroPay	refund	Carol	50.00	Success (45.00 EUR)

Expected client output (wording may vary):

```
Legacy Bank | pay    | Alice | $25.50   -> SUCCESS
QuickPay    | pay    | Bob   | $100.00  -> SUCCESS
EuroPay     | pay    | Carol | $50.00   -> SUCCESS
Legacy Bank | pay    | Dave  | $1500.00 -> FAILED
QuickPay    | refund | Bob   | $100.00  -> SUCCESS
EuroPay     | refund | Carol | $50.00   -> SUCCESS
```

Deliverables and Demonstration

Submit:

1. A short document with your answers to Task 1.
2. Your UML class diagram and sequence diagram (Task 2).
3. Your source code (Task 3).

You will **demonstrate your program in class by running the code** and briefly explain your design. Your instructor may ask you to add a fourth payment service live. A program that cannot be run in class receives no marks for Task 3.